

消息中心微服务化完整实现手册

消息中心微服务化完整实现手册

本文档整合消息中心独立微服务的架构设计、交互流程、接口规范、可复用代码、部署方案与落地步骤，单文档一站式查阅，基于 Spring Boot + Spring Cloud 技术栈实现。

一、架构总览与职责边界

1.1 核心定位

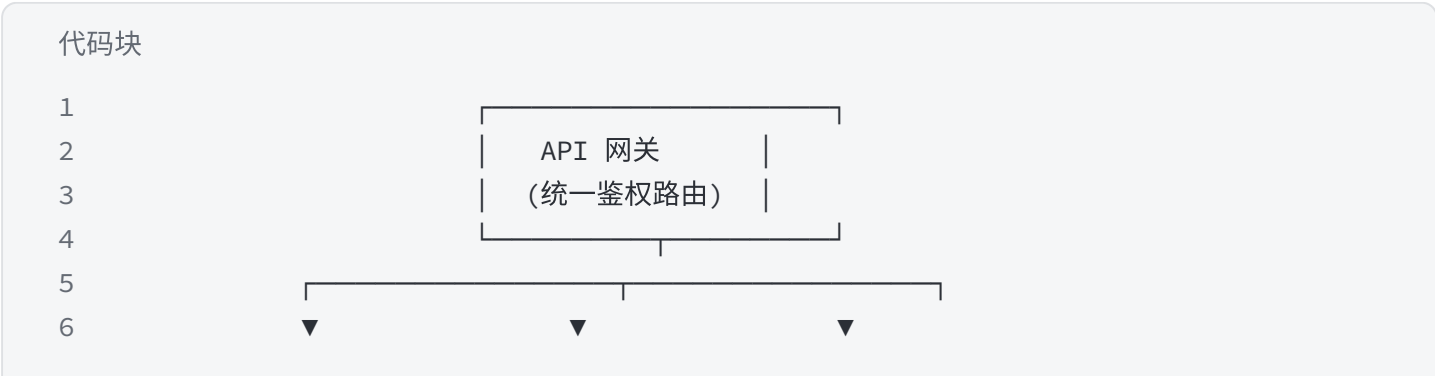
消息中心作为全校统一的独立消息中台（Notification Service），与课表预约、教学管理、学生管理等业务系统完全解耦，独立部署、独立扩缩容、独立迭代，负责全渠道消息投递与全生命周期管理。

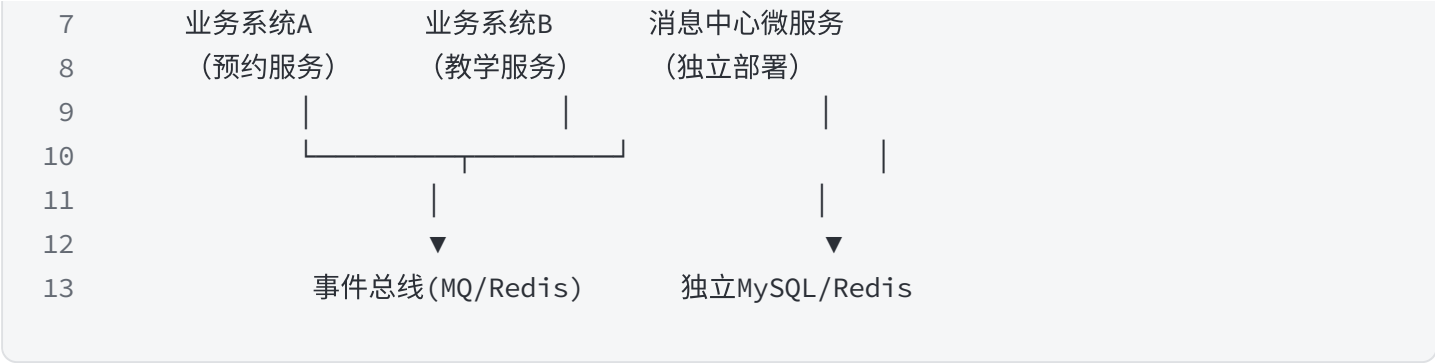
1.2 职责划分

角色	职责范围
消息中心微服务	全渠道消息投递（WebSocket站内推送、离线补偿、批量广播）；消息全生命周期管理（存储、分类、已读/收藏/删除、归档）；推送通道运维（连接管理、流量控制、优先级调度、投递追踪）；基础能力（模板、敏感词过滤、AES加密存储、统计审计）；对外提供标准化接口
上游业务系统	业务规则判断（触发时机、接收对象、内容组装）；按约定格式组装消息参数；调用消息中心接口或投递业务事件；接收消息状态回调并更新自身业务单据

边界原则：业务系统不感知推送技术细节，消息中心不感知业务逻辑。

1.3 部署拓扑





二、四种核心交互模式

2.1 同步API调用模式

- **适用场景：**单条精准发送、高可靠业务通知（如预约成功、请假提交回执）
- **交互流程：**业务系统在业务流程节点同步调用消息中心发送接口 → 消息中心接收后立即落库 → 异步执行推送 → 同步返回消息ID
- **特点：**调用后立即返回，投递异步执行，业务线程不阻塞

2.2 异步事件驱动模式

- **适用场景：**全年级批量通知、高并发触发场景（如排期更新、作业发布）
- **交互流程：**业务系统发布业务事件到消息队列 → 消息中心订阅事件 → 自动转换成消息投递
- **特点：**完全解耦，削峰填谷，支持批量消费，业务系统发完事件即可继续执行

2.3 状态回调模式

- **适用场景：**需要确认用户已读的重要通知、请假审批已读回执、作业查看追踪
- **交互流程：**消息中心状态变更（投递成功/失败、用户已读、用户点击） → 异步调用业务系统回调地址 → 业务系统更新业务单据状态
- **特点：**异步非阻塞，支持失败重试、幂等校验

2.4 前端直连模式

- **适用场景：**实时消息接收、未读数实时更新、多端状态同步
- **交互流程：**前端浏览器直接通过网关与消息中心建立WebSocket连接 → 接收实时推送、同步消息状态
- **特点：**消息流不占用业务系统资源，业务系统无感知

三、标准化接口协议

3.1 消息发送接口（消息中心提供）

- **接口地址：** `POST /api/internal/v1/messages/send`

- 鉴权方式：服务间内部API密钥 + 请求签名
- 请求体格式：

代码块

```
1  {
2    "bizCode": "APPOINTMENT_CONFIRM",
3    "receiverType": "USER_ID",
4    "receiverIds": ["1001", "1002"],
5    "messageType": "SYSTEM_NOTICE",
6    "priority": "NORMAL",
7    "title": "预约成功通知",
8    "content": "您的课程预约已确认，请准时参加。",
9    "jumpUrl": "/appointment/detail/12345",
10   "bizId": "appointment_12345",
11   "sender": "课程预约系统",
12   "callbackUrl": "https://appointment-service/api/callback/message-status"
13 }
```

3.2 批量发送接口

- 接口地址： POST /api/internal/v1/messages/batch
- 扩展参数：支持按用户角色、年级、班级等维度群发， receiverType 可选 ROLE / GRADE / ALL

3.3 事件驱动契约（队列交互）

标准事件格式，所有业务系统遵循同一规范：

代码块

```
1  {
2    "eventId": "evt_8f3a2c1e",
3    "eventType": "APPOINTMENT_BOOKED",
4    "source": "appointment-service",
5    "timestamp": 1710000000000,
6    "data": {
7      "userId": "1001",
8      "bizId": "appointment_12345",
9      "title": "预约成功通知",
10     "content": "您的课程预约已确认，请准时参加。",
11     "jumpUrl": "/appointment/detail/12345"
12   }
13 }
```

3.4 状态回调协议（消息中心 → 业务系统）

- **回调时机：**投递成功、投递失败、用户已读、用户点击跳转
- **重试策略：**失败后指数退避重试，最多3次
- **回调格式：**

代码块

```
1  {
2    "messageId": "msg_7d2e9f0a",
3    "bizId": "appointment_12345",
4    "bizCode": "APPOINTMENT_CONFIRM",
5    "status": "READ",
6    "userId": "1001",
7    "timestamp": 1710000500000
8  }
```

四、核心可复用代码实现

4.1 消息中心服务端实现

4.1.1 消息实体类

代码块

```
1  package com.notification.entity;
2
3  import com.baomidou.mybatisplus.annotation.IdType;
4  import com.baomidou.mybatisplus.annotation.TableId;
5  import com.baomidou.mybatisplus.annotation.TableName;
6  import lombok.Data;
7  import java.time.LocalDateTime;
8
9  @Data
10 @TableName("message")
11 public class Message {
12     @TableId(type = IdType.ASSIGN_ID)
13     private String messageId;
14
15     private String bizCode;
16     private String bizId;
17     private String sender;
18     private String messageType;
19     private String priority;
20     private String title;
```

```
21     private String content;
22     private String jumpUrl;
23     private String callbackUrl;
24     private Integer isBroadcast;
25     private LocalDateTime sendTime;
26     private LocalDateTime createTime;
27     private LocalDateTime updateTime;
28 }
```

4.1.2 消息发送DTO

代码块

```
1  package com.notification.dto;
2
3  import lombok.Data;
4  import java.util.List;
5
6  @Data
7  public class MessageSendDTO {
8      private String bizCode;
9      private String receiverType;
10     private List<String> receiverIds;
11     private String messageType;
12     private String priority;
13     private String title;
14     private String content;
15     private String jumpUrl;
16     private String bizId;
17     private String sender;
18     private String callbackUrl;
19 }
```

4.1.3 消息发送核心Service

代码块

```
1  package com.notification.service;
2
3  import com.notification.dto.MessageSendDTO;
4  import com.notification.entity.Message;
5  import com.notification.entity.UserMessageInbox;
6  import com.notification.mapper.MessageMapper;
7  import com.notification.mapper.UserMessageInboxMapper;
8  import com.notification.util.AesGcmCryptoUtil;
9  import lombok.RequiredArgsConstructor;
```

```
10 import org.springframework.data.redis.core.StringRedisTemplate;
11 import org.springframework.stereotype.Service;
12 import org.springframework.transaction.annotation.Transactional;
13
14 import java.time.LocalDateTime;
15 import java.util.List;
16 import java.util.UUID;
17
18 @Service
19 @RequiredArgsConstructor
20 public class MessageService {
21
22     private final MessageMapper messageMapper;
23     private final UserMessageInboxMapper inboxMapper;
24     private final StringRedisTemplate redisTemplate;
25     private final WebSocketPushService websocketPushService;
26
27     @Value("${app.aes-gcm.secret}")
28     private String aesSecret;
29
30     @Transactional(rollbackFor = Exception.class)
31     public String sendMessage(MessageSendDTO dto) {
32         // 1. 消息持久化 (内容加密存储)
33         Message message = new Message();
34         String messageId = "msg_" + UUID.randomUUID().toString().replace("-",
35         "").substring(0, 16);
36         message.setMessageId(messageId);
37         message.setBizCode(dto.getBizCode());
38         message.setBizId(dto.getBizId());
39         message.setSender(dto.getSender());
40         message.setMessageType(dto.getMessageType());
41         message.setPriority(dto.getPriority());
42         message.setTitle(AesGcmCryptoUtil.encrypt(dto.getTitle(), aesSecret));
43         message.setContent(AesGcmCryptoUtil.encrypt(dto.getContent(),
44         aesSecret));
45         message.setJumpUrl(dto.getJumpUrl());
46         message.setCallbackUrl(dto.getCallbackUrl());
47         message.setIsBroadcast(0);
48         message.setSendTime(LocalDateTime.now());
49         message.setCreateTime(LocalDateTime.now());
50         message.setUpdateTime(LocalDateTime.now());
51         messageMapper.insert(message);
52
53         // 2. 写入用户收件箱索引
54         List<String> receiverIds = dto.getReceiverIds();
55         for (String userId : receiverIds) {
56             UserMessageInbox inbox = new UserMessageInbox();
```

```

55         inbox.setUserId(userId);
56         inbox.setMessageId(messageId);
57         inbox.setIsRead(0);
58         inbox.setIsStarred(0);
59         inbox.setIsDeleted(0);
60         inbox.setCreatedAt(LocalDateTime.now());
61         inboxMapper.insert(inbox);
62
63         // 3. 未读数+1
64         redisTemplate.opsForValue().increment("unread:count:" + userId);
65     }
66
67     // 4. 异步WebSocket推送
68     websocketPushService.pushToUsers(receiverIds, message);
69
70     return messageId;
71 }
72 }

```

4.1.4 内部发送接口Controller

代码块

```

1  package com.notification.controller;
2
3  import com.notification.dto.MessageSendDTO;
4  import com.notification.service.MessageService;
5  import com.notification.vo.ResultVO;
6  import lombok.RequiredArgsConstructor;
7  import org.springframework.web.bind.annotation.*;
8
9  @RestController
10 @RequestMapping("/api/internal/v1/messages")
11 @RequiredArgsConstructor
12 public class MessageInternalController {
13
14     private final MessageService messageService;
15
16     @PostMapping("/send")
17     public ResultVO<String> sendMessage(@RequestBody MessageSendDTO dto) {
18         String messageId = messageService.sendMessage(dto);
19         return ResultVO.success(messageId);
20     }
21
22     @GetMapping("/{messageId}/status")

```

```

23     public ResultVO<MessageStatusVO> getMessageStatus(@PathVariable String
        messageId) {
24         return ResultVO.success(messageService.getMessageStatus(messageId));
25     }
26 }

```

4.1.5 WebSocket配置与推送服务

代码块

```

1  package com.notification.config;
2
3  import org.springframework.context.annotation.Configuration;
4  import org.springframework.messaging.simp.config.MessageBrokerRegistry;
5  import
    org.springframework.web.socket.config.annotation.EnableWebSocketMessageBroker;
6  import org.springframework.web.socket.config.annotation.StompEndpointRegistry;
7  import
    org.springframework.web.socket.config.annotation.WebSocketMessageBrokerConfigurer;
8
9  @Configuration
10 @EnableWebSocketMessageBroker
11 public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {
12
13     @Override
14     public void configureMessageBroker(MessageBrokerRegistry config) {
15         config.enableSimpleBroker("/topic", "/user");
16         config.setApplicationDestinationPrefixes("/app");
17         config.setUserDestinationPrefix("/user");
18     }
19
20     @Override
21     public void registerStompEndpoints(StompEndpointRegistry registry) {
22         registry.addEndpoint("/ws/notification")
23             .setAllowedOriginPatterns("*")
24             .withSockJS();
25     }
26 }

```

代码块

```

1  package com.notification.service;
2
3  import lombok.RequiredArgsConstructor;
4  import org.springframework.messaging.simp.SimpMessagingTemplate;

```



```

5  import org.springframework.stereotype.Service;
6
7  import java.util.List;
8
9  @Service
10 @RequiredArgsConstructor
11 public class WebSocketPushService {
12
13     private final SimpMessagingTemplate messagingTemplate;
14
15     public void pushToUsers(List<String> userIds, Object message) {
16         for (String userId : userIds) {
17             messagingTemplate.convertAndSendToUser(userId,
18                 "/queue/notifications", message);
19         }
20
21     public void broadcast(Object message) {
22         messagingTemplate.convertAndSend("/topic/notifications", message);
23     }
24 }

```

4.1.6 业务事件消费者（异步事件模式）

代码块

```

1  package com.notification.consumer;
2
3  import com.alibaba.fastjson.JSON;
4  import com.notification.dto.MessageSendDTO;
5  import com.notification.event.BusinessEvent;
6  import com.notification.service.MessageService;
7  import lombok.RequiredArgsConstructor;
8  import org.springframework.data.redis.stream.StreamListener;
9  import org.springframework.stereotype.Component;
10
11 import java.util.Collections;
12
13 @Component
14 @RequiredArgsConstructor
15 public class BusinessEventConsumer implements StreamListener<String,
16     ObjectRecord<String, String>> {
17
18     private final MessageService messageService;
19
20     @Override

```

```

20     public void onMessage(ObjectRecord<String, String> record) {
21         String value = record.getValue();
22         BusinessEvent event = JSON.parseObject(value, BusinessEvent.class);
23
24         // 事件转消息
25         MessageSendDTO dto = new MessageSendDTO();
26         dto.setBizCode(event.getEventType());
27         dto.setReceiverType("USER_ID");
28
29         dto.setReceiverIds(Collections.singletonList(event.getData().getUserId()));
30         dto.setMessageType("SYSTEM_NOTICE");
31         dto.setPriority("NORMAL");
32         dto.setTitle(event.getData().getTitle());
33         dto.setContent(event.getData().getContent());
34         dto.setJumpUrl(event.getData().getJumpUrl());
35         dto.setBizId(event.getData().getBizId());
36         dto.setSender(event.getSource());
37
38         messageService.sendMessage(dto);
39     }

```

4.2 业务系统客户端实现

4.2.1 OpenFeign调用客户端

代码块

```

1  package com.business.client;
2
3  import com.business.vo.ResultVO;
4  import org.springframework.cloud.openfeign.FeignClient;
5  import org.springframework.web.bind.annotation.PostMapping;
6  import org.springframework.web.bind.annotation.RequestBody;
7
8  import java.util.Map;
9
10 @FeignClient(name = "notification-service", path = "/api/internal/v1/messages")
11 public interface NotificationClient {
12
13     @PostMapping("/send")
14     ResultVO<String> sendMessage(@RequestBody Map<String, Object> messageDTO);
15 }

```

4.2.2 业务系统调用示例

代码块

```
1  package com.business.service;
2
3  import com.business.client.NotificationClient;
4  import lombok.RequiredArgsConstructor;
5  import org.springframework.stereotype.Service;
6
7  import java.util.HashMap;
8  import java.util.List;
9  import java.util.Map;
10
11 @Service
12 @RequiredArgsConstructor
13 public class AppointmentService {
14
15     private final NotificationClient notificationClient;
16
17     public void confirmAppointment(String userId, String appointmentId) {
18         // 业务逻辑：确认预约
19         // ...
20
21         // 调用消息中心发送通知
22         Map<String, Object> message = new HashMap<>();
23         message.put("bizCode", "APPOINTMENT_CONFIRM");
24         message.put("receiverType", "USER_ID");
25         message.put("receiverIds", List.of(userId));
26         message.put("messageType", "SYSTEM_NOTICE");
27         message.put("priority", "NORMAL");
28         message.put("title", "预约成功通知");
29         message.put("content", "您的课程预约已确认，请准时参加。");
30         message.put("jumpUrl", "/appointment/detail/" + appointmentId);
31         message.put("bizId", "appointment_" + appointmentId);
32         message.put("sender", "课程预约系统");
33         message.put("callbackUrl", "https://appointment-
34             service/api/callback/message-status");
35
36         notificationClient.sendMessage(message);
37     }
38 }
```

4.2.3 状态回调接口（业务系统提供）

代码块

```
1  package com.business.controller;
2
```

```

3  import com.business.vo.ResultVO;
4  import org.springframework.web.bind.annotation.*;
5
6  @RestController
7  @RequestMapping("/api/callback")
8  public class MessageCallbackController {
9
10     @PostMapping("/message-status")
11     public ResultVO<Void> messageStatusCallback(@RequestBody Map<String,
Object> callback) {
12         String bizId = (String) callback.get("bizId");
13         String status = (String) callback.get("status");
14         String userId = (String) callback.get("userId");
15
16         // 更新业务单据的通知状态
17         // appointmentService.updateNoticeStatus(bizId, status);
18
19         return ResultVO.success();
20     }
21 }

```

4.3 前端接入代码

代码块

```

1  import SockJS from 'sockjs-client';
2  import Stomp from 'stompjs';
3
4  let stompClient = null;
5  let unreadCount = 0;
6
7  // 建立WebSocket连接
8  export function connectNotification(userId, token) {
9      const socket = new SockJS('/ws/notification');
10      stompClient = Stomp.over(socket);
11
12      stompClient.connect({ Authorization: `Bearer ${token}` }, () => {
13          // 订阅个人消息队列
14          stompClient.subscribe(`/user/queue/notifications`, (message) => {
15              const msg = JSON.parse(message.body);
16              handleNewMessage(msg);
17              unreadCount++;
18              updateUnreadBadge();
19          });
20
21          // 订阅未读数同步

```

```

22         stompClient.subscribe(`/user/queue/unread-count`, (message) => {
23             unreadCount = parseInt(message.body);
24             updateUnreadBadge();
25         });
26     });
27 }
28
29 // 处理新消息
30 function handleNewMessage(msg) {
31     console.log('收到新消息: ', msg);
32     // 渲染消息提示、更新消息列表
33 }
34
35 // 更新未读角标
36 function updateUnreadBadge() {
37     document.getElementById('unread-badge').textContent = unreadCount;
38 }
39
40 // 获取消息列表
41 export async function getMessageList(page = 1, size = 20) {
42     const res = await fetch(`/api/notification/v1/users/inbox?
43     page=${page}&size=${size}`);
44     return res.json();
45 }
46
47 // 标记已读
48 export async function markAsRead(messageId) {
49     await fetch(`/api/notification/v1/users/messages/${messageId}/read`, {
50         method: 'POST' });
51 }

```

五、部署与网关配置

5.1 Spring Cloud Gateway 路由配置

代码块

```

1  spring:
2    cloud:
3      gateway:
4        routes:
5          # 消息中心REST接口路由
6          - id: notification-rest
7            uri: lb://notification-service
8            predicates:
9              - Path=/api/notification/**

```

```

10         filters:
11             - StripPrefix=2
12
13         # 消息中心WebSocket路由
14         - id: notification-ws
15           uri: lb:ws://notification-service
16         predicates:
17             - Path=/ws/notification/**

```

5.2 Nacos服务发现配置

消息中心服务配置：

代码块

```

1  spring:
2      application:
3          name: notification-service
4      cloud:
5          nacos:
6              discovery:
7                  server-addr: ${NACOS_ADDR:127.0.0.1:8848}
8                  namespace: production
9              config:
10                 server-addr: ${NACOS_ADDR:127.0.0.1:8848}
11                 namespace: production
12                 file-extension: yaml

```

5.3 独立数据配置

消息中心使用独立的MySQL和Redis实例，配置独立管理：

代码块

```

1  spring:
2      datasource:
3          driver-class-name: com.mysql.cj.jdbc.Driver
4          url: jdbc:mysql://${DB_HOST}:${DB_PORT}/notification_db?
5              useUnicode=true&characterEncoding=utf8
6          username: ${DB_USER}
7          password: ${DB_PASSWORD}
8
9      data:
10         redis:
11             host: ${REDIS_HOST}
12             port: ${REDIS_PORT}

```

```
12     password: ${REDIS_PASSWORD}
13     database: 1
```

六、平滑演进落地步骤

第一步：接口抽象与抽取

1. 将现有消息中心模块从业务代码中抽离为独立Maven模块，定义标准接口
2. 业务系统通过接口调用，不直接依赖内部实现，为拆分做准备
3. 梳理所有消息触发点，统一收口到接口层

第二步：独立部署，双轨运行

1. 将消息模块打包为独立Spring Boot应用，独立部署
2. 业务系统同时保留本地调用和远程调用两种方式，通过配置切换
3. 先将非核心消息切到独立服务，验证稳定性后逐步全量切换
4. 验证数据一致性、推送可靠性、接口兼容性

第三步：完整解耦，事件驱动

1. 下线业务系统本地消息模块，全部切换为远程调用
2. 引入事件总线，将高并发、批量场景切换为事件驱动模式
3. 完善监控、告警、链路追踪，完成完整微服务治理
4. 接入更多业务系统，沉淀统一消息中台能力

七、生产级注意事项

1. **幂等性保障**：所有发送接口、回调接口都必须实现幂等，通过消息ID、业务ID去重，避免重复发送、重复回调
2. **一致性保证**：消息发送与业务提交保持最终一致，采用“业务事务提交后再发消息”+“失败重试”模式，避免业务回滚但消息已发
3. **限流与熔断**：对每个业务系统设置发送量阈值，避免单一业务突增流量打垮消息中心；调用端配置熔断降级，消息中心不可用时不影响主业务流程
4. **安全隔离**：内部接口不对外网开放，服务间调用采用内部鉴权；敏感消息内容加密传输，密钥独立管理
5. **链路追踪**：接入SkyWalking等链路追踪工具，透传traceId，实现从业务触发到消息投递的全链路可追踪

6. **性能优化**：用户收件箱按用户ID分表，热点未读数缓存到Redis；批量消息采用分片异步处理，避免长事务
7. **高可用部署**：消息中心多节点部署，无状态水平扩展；Redis集群、MySQL主从保证数据层高可用

（注：部分内容可能由 AI 生成）